Quantstamp

Nillion - NilAv

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Governance
Timeline	2026-01-05 through 2026-01-09
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• NillionNetwork/niluv #af92088
Auditors	• Darren Jensen Auditing Engineer • Jennifer Wu Auditing Engineer • Rabib Islam Senior Auditing Engineer

Documentation quality	Medium
Test quality	Medium
Total Findings	12 Fixed: 8 Acknowledged: 2 Mitigated: 2
High severity findings	0
Medium severity findings	0
Low severity findings	7 Fixed: 5 Mitigated: 2
Undetermined severity findings	0
Informational findings	5 Fixed: 3 Acknowledged: 2

Summary of Findings

The NilAv protocol implements the core on-chain components of a stake-weighted heartbeat verification and enforcement system with modular slashing, jailing, rewards, and emissions. It is designed for a proof-of-stake environment where operators stake an ERC20 token, are randomly selected into committees, vote on “heartbeats” and can be rewarded or punished based on outcomes.

Overall, the contracts are well-written and architected. However, the audit has identified several low/informational vulnerabilities and recommendations related to untrusted input, insufficient bounds, and potential denial-of-service conditions within the audited contracts.

One concern is the unchecked `committeeSize` in the `JailingPolicy.recordRound()` function, which can be manipulated by external callers, potentially leading to a permanent disabling of proofless jailing functionality (`NIL-1`). Similarly, the `StakingOperators` contract suffers from an unbounded `_activeOperators` array, allowing for potential gas and memory denial-of-service attacks (`NIL-7`); instituting a hard cap on the number of operators or enforcing a higher minimum stake is recommended.

Additional vulnerabilities include a lack of consent from operators when staking on their behalf, which can lead to unintended dependencies on the staker; a suggested remedy involves implementing explicit consent mechanisms for initial stakership (`NIL-2`). Further, the auditing process has flagged issues like the potential for ETH to become stranded within the `EmissionsController` due to the absence of a withdrawal system, emphasizing the need for a recovery function or automatic refunds for zero-value transactions.

Each of these findings emphasizes the need for stringent input validation, access control, and monitoring mechanisms to support the protocol's integrity and usability as the development evolves. Implementing the recommended changes will enhance overall security and operational reliability.

Fix-Review Update 2026-01-16:
The client has effectively addressed several vulnerabilities and suggestions in their code update. Specifically, vulnerabilities NIL-1, NIL-4, NIL-5, NIL-6, NIL-7, NIL-8, NIL-9, and NIL-12 were fixed by implementing various safeguards and enhancements, such as improved jailing policies, reward distribution mechanisms, and error handling to bolster overall security and functionality. Vulnerabilities NIL-2 and NIL-3 were mitigated through measures that establish operator approvals and bond requirements to minimize risks, though they do not fully eliminate the vulnerabilities as the fixes are not definitive. Additionally, vulnerabilities NIL-10 and NIL-11 were acknowledged rather than fixed, with the client opting to document specific operational requirements and limitations instead. Suggestions S1, S2, S3, and S4 were mostly successfully implemented, enhancing code quality and operational efficiency. Overall, the majority of vulnerabilities have been resolved, significantly improving the security posture of the client's code.

ID	DESCRIPTION	SEVERITY	STATUS
NIL-1	Untrusted `committeeSize` Can Permanently Disable Proofless Jailing Path	Low	Fixed
NIL-2	First Staker Can Squat Operator Address	Low	Mitigated
NIL-3	Permissionless Heartbeat Submission Can Be Used to Jail Non-Voters and Skew Committee Selection	Low	Mitigated
NIL-4	`ETH` Can Become Stuck when Epoch Emission Is Zero	Low	Fixed
NIL-5	Operators Are Incentivized to Collude and Always Vote Valid	Low	Fixed
NIL-6	Batch Jailing Can Be Griefed via Front-Running Already-Enforced Operators	Low	Fixed
NIL-7	Unbounded Active Operator Set Can Cause Gas/Memory DoS During Committee Selection	Low	Fixed
NIL-8	Slashing Callback Can Block Round Finalization via `OOG` /return-data Bomb	Informational	Fixed
NIL-9	Staker Binding Cannot Be Cleared when Operator Has Zero Stake and No Tranches	Informational	Fixed
NIL-10	Unbonding Tranche Limit May Prevent Further Unstake Requests	Informational	Acknowledged
NIL-11	`RewardPolicy` Is Incompatible with Deflationary Tokens	Informational	Acknowledged
NIL-12	Unsafe Casting Can Cause Integer Truncation	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following

    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/NillionNetwork/niluv`

Included Paths: `contracts/src`

Operational Considerations

- Committee selection randomness is predictable/grindable, allowing manipulation by transaction timing or snapshot selection. The randomness seed in `WeightedCommitteeSelector` is derived from `blockhash(snapshotId)` or falls back to `prevrandao`, combined with other predictable elements. As acknowledged in the documentation, this creates a "grindable randomness surface.". A sophisticated and motivated actor who submits heartbeats could theoretically time their submissions to influence the block-dependent seed, thereby increasing the probability of a favorable committee being selected.
- The unbonding tranche limit (`MAX_TRANCHES_PER_OPERATOR = 32`) can temporarily prevent additional unstake requests until existing tranches are withdrawn.
- Slashed tokens are permanently burned and cannot be recovered, redirected to a treasury, or used for insurance purposes.
- The owner can replace critical protocol modules with arbitrary contracts via `HeartbeatManager.setConfig()` and `ProtocolConfig.setModules()`, creating governance and key-compromise risk; use of a multisig is strongly recommended.
- In `ProtocolConfig` an admin can call `setModules()` function to update critical protocol modules which are replaced instantly without any timelock or delay.
- Solidity pragma is unspecified/floating, which may lead to inconsistent compiler behavior across builds.
- The owner can prevent reward distribution to valid voters by calling `HeartbeatManager.abandonRewardDistribution()`.
- In `EmissionsController`, if `mintAndBridgeNextEpoch()` is not called during an epoch, emissions for that epoch will not be bridged to L2.
- An unlimited ERC20 allowance is granted to the bridge, enabling full fund drain if the bridge is compromised or behaves maliciously.
- In `EmissionsController`, the epoch schedule can become permanently blocked if a scheduled emission exceeds the remaining `globalMintCap`.
- ETH sent to `EmissionsController` can become permanently stranded due to payable functions without an ETH recovery mechanism.
- The active operator set can become stale after governance changes (e.g., `minOperatorStake` updates) unless operators are explicitly reconciled via `pokeActive*`.
- Reward streaming behavior depends on precise timing of deposits and `sync()` calls, making reward availability sensitive to operational timing.
- Permissionless maintenance functions (e.g., `sync()`, `pokeActive()`) assume active off-chain keepers for timely state reconciliation and liveness.
- Committee members can vote without requiring to re-evaluate active eligibility (e.g. satisfies min stake, not jailed etc) at vote time.
- Reward allocation depends on distribution timing when budget is uncapped. Since no budget is reserved at finalization, the amount allocated to a round depends on when `HeartbeatManager.distributeRewards()` is executed and, when multiple rounds are pending distribution, the order in which rounds are processed, rather than on the round's finalization time.
- `RewardPolicy` solvency has potential for freezing rewards. If the `RewardPolicy` balance ever drops below its obligations an `accountingFrozen` flag is set which immediately freezes rewards until the Owner manually restores the contract's solvency.
- Caller chosen snapshot can bias committee selection and voting power. `HeartbeatManager.submitHeartbeat()` enables a submitter to select a snapshot where their stake share is higher and/or where the deterministic randomness seed yields a favorable committee, effectively increasing the portion of committee voting power they control for that round.
- The `EmissionsController` is fundamentally dependent on an external L1 bridge contract to move minted tokens to L2. The security and liveness of the emissions process are therefore tied to the correctness, security, and operational status of this third-party bridge.

Key Actors And Their Capabilities

Owner / Governance

- Update protocol parameters and module addresses via `ProtocolConfig`.
- Pause/unpause core components and abandon reward distribution.
- Control emission schedules, reward settings, and bridge configuration.

- Withdraw excess funds and clear accounting freezes.

Staking Operators

- Register as operators once minimum stake is met.
- Participate in committees and submit signed verdicts.
- Earn rewards or be jailed/slashed based on behavior.
- Reactivate after jail expiry.

Slasher Role (`SLASHER_ROLE`)

- Highly specialized and destructive role, designed to be held by the `JailingPolicy` contract or a trusted administrator.
- This role is responsible for enforcing penalties against misbehaving operators.
- Can burn a specified amount of an operator's staked tokens.
- Can deactivate an operator for a set period, preventing them from participating in committees.

Stakers / Delegators

- Stake tokens on behalf of an operator address.
- Exclusively control unstaking and withdrawals for that operator.

Committee Members

- Vote on heartbeat validity during assigned rounds.
- Influence outcomes based on snapshotted stake.

Keepers / Off-Chain Automation

- Call maintenance functions (e.g., `sync`, `pokeActive`, escalation, jailing).
- Maintain protocol liveness.

L1 Bridge

- Bridge minted tokens from L1 to L2.
- Custody funds under unlimited allowance.

Permissionless Actors

- Interact with public functions and batch operations (heartbeats, jailing, stake, liveness etc).
- Submit heartbeats and initiate verification rounds.

Findings

NIL-1 Untrusted `committeeSize` Can Permanently Disable Proofless Jailing Path

Low · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `57b7c9d`.
The client provided the following explanation:

> JailingPolicy now records committee size from HeartbeatManager to prevent poisoned inputs from disabling jailing without proof.

File(s) affected: `contracts/src/JailingPolicy.sol`

Description: The `JailingPolicy.recordRound()` function stores a `committeeSize` value that is supplied externally and not authenticated against the canonical round data maintained by `HeartbeatManager`. While `onRoundFinalized()` is intended to populate this value, `recordRound()` is `public` and can be called permissionlessly by any address once a round is finalized.

Since `recordRound()` is idempotent, the first caller permanently determines the stored `committeeSize`. Subsequent legitimate calls from `HeartbeatManager` will no-op and cannot correct an incorrect value.

The stored `committeeSize` is later used as a strict precondition in `enforceJailFromMembers()`, which requires `sortedMembers.length == rr.committeeSize`. If the value is poisoned, this function will always revert, permanently disabling the proofless jailing path for the affected round.

Recommendation: Consider removing the external `committeeSize` parameter entirely from `recordRound()` and source the canonical committee size directly from `HeartbeatManager`. This can be achieved by extending `getRoundForPolicy()` to return the committee size, or by adding a dedicated getter for it.

Alternatively, restrict `recordRound()` so it can only be called by `HeartbeatManager`, ensuring all recorded data is authoritative and cannot be poisoned.

NIL-2 First Staker Can Squat Operator Address

Low · Mitigated

Update

Marked as "Mitigated" by the client.
Addressed in: `5158fa9`.

The client provided the following explanation:

> Added optional operator approval for initial binding and require the initial stake to meet the minimum active-operator stake to reduce squatting risk.

File(s) affected: `contracts/src/StakingOperators.sol`

Description: `StakingOperators.stakeTo()` binds `operatorStaker[operator] = msg.sender` on first use. If an operator address is known in advance or intended to be used with a specific staker, a third party can preemptively stake a dust amount to that operator address and claim the staker binding.

While the operator address itself can still call `StakingOperators.registerOperator()`, the operator then becomes dependent on an unintended staker for all stake management actions (including additional staking, unstaking, and withdrawals). This prevents the intended staker from staking to or operating that operator address (via `DifferentStaker`), resulting in a denial-of-service condition for the intended operator–staker pairing.

Recommendation: Consider whether the initial operator–staker binding is intended to be permissionless. If not, consider introducing an explicit authorization mechanism for first-time bindings to prevent unintended staker assignment. Depending on the intended design, this could include requiring operator approval for the initial binding (e.g., via an off-chain signature), restricting first-time binding to cases where the operator initiates the stake, or namespacing operators so that bindings are scoped per staker rather than globally (e.g., deriving an operator identifier from `(staker, operator)`).

NIL-3 Permissionless Heartbeat Submission Can Be Used to Jail Non-Voters and Skew Committee Selection

• Low Mitigated

> **Update**
> Marked as "Mitigated" by the client.
> Addressed in: `f743293`.
> The client provided the following explanation:
>
> > Added heartbeat bond requirement and gated non-voter jailing to reduce spam-based griefing risk.

File(s) affected: `contracts/src/HeartbeatManager.sol`, `contracts/src/JailingPolicy.sol`, `contracts/src/StakingOperators.sol`

Description: `HeartbeatManager.submitHeartbeat()` is permissionless and starts a new verification round upon submission. Since submitting a heartbeat requires no fee or bond beyond gas, an attacker can create a large number of concurrent heartbeat instances (distinct `heartbeatKey`s) at relatively low cost.

`JailingPolicy` treats any committee member who does not respond as jailable (`_isJailable()` returns `true` for non-voters regardless of the round outcome). Therefore, an attacker can increase the probability that honest committee members miss at least one vote (e.g., due to operational load or liveness issues) and then permissionlessly enforce jailing via `enforceJail` / `enforceJailFromMembers`. Because jailed operators are removed from `getActiveOperators()`, repeated jailing can shrink the active set and increase the attacker's expected committee representation over time.

Exploit Scenario:

1. The attacker registers multiple minimally staked operators.
2. The attacker submits many heartbeat instances (distinct `heartbeatKey`s) to create a high volume of active rounds.
3. Some honest committee members fail to respond before deadlines due to operational load or liveness issues.
4. The attacker enforces jailing for non-voters, temporarily removing them from the active set.
5. With fewer honest operators eligible, attacker-controlled operators represent a larger fraction of the active set and are more likely to be selected into subsequent committees.

Recommendation: Consider introducing friction or authorization for heartbeat initiation (e.g., rate limiting per submitter, a fee/bond refundable upon successful participation, or restricting submission to authenticated sources). Additionally, consider constraining non-voter jailing to reduce griefing (e.g., only allowing non-voter penalties when a minimum participation threshold is met, or when the heartbeat is from an authorized source).

NIL-4 `ETH` Can Become Stuck when Epoch Emission Is Zero

• Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `117728a`.

The client provided the following explanation:

> Revert when ETH is sent alongside a zero-emission epoch to prevent trapped value.

File(s) affected: `contracts/src/EmissionsController.sol`

Description: `mintAndBridgeNextEpoch{value: msg.value}` forwards `msg.value` to `bridge.depositERC20To()` only when `amount != 0`. If the scheduled emission for an epoch is zero, the function succeeds but does not call the bridge, leaving the ETH trapped in the `EmissionsController` contract with no withdrawal mechanism.

Recommendation: Revert when `msg.value != 0 && amount == 0`, or add an owner-only ETH rescue function (and/or always forward value via a zero-amount bridge call if supported).

NIL-5 Operators Are Incentivized to Collude and Always Vote Valid

• Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `1787124`.
The client provided the following explanation:

> Reward distribution now supports invalid-threshold outcomes so correct voters can be rewarded for invalid results.

Description: Like most blockchain consensus protocols, the current protocol for heartbeat verification relies on a quorum of operators being met in order to authenticate heartbeats. However, operators in most consensus protocols are rewarded regardless of the type of vote they cast, so long as they align with the honest majority.

In the current consensus protocol, operators are only rewarded when they cast a vote that the heartbeat under question is valid. This creates an incentive for operators to always vote valid, as there is no chance of receiving a reward otherwise.

This further creates an incentive for operators to collude to always vote valid, as this maximizes the potential reward for each operator.

Recommendation: Consider using a consensus protocol that rewards operators for honest participation rather than one that only rewards votes that heartbeats are valid.

NIL-6 Batch Jailing Can Be Griefed via Front-Running Already-Enforced Operators

• Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `5c82292`.
The client provided the following explanation:

> Batch jailing now skips already enforced operators and uses explicit length-mismatch errors to avoid griefing and improve error hygiene.

File(s) affected: `contracts/src/JailingPolicy.sol`

Description: In `JailingPolicy` the `enforceJailMany()` function performs batch jailing by iterating over operators and calling `enforceJail()` for each. If any individual call reverts (for example due to `AlreadyEnforced`), the entire batch reverts.

Since `enforceJail()` reverts when an operator has already been enforced, the batch function has no per-item fault tolerance. This makes the batch vulnerable to front-running or race conditions where one operator in the batch becomes enforced before the batch transaction is mined. As a result, batch jailing can be disrupted even though all other operators in the batch are valid and jailable.

Exploit Scenario:

1. A user submits `enforceJailMany()` with a list of operators to be jailed.
2. An attacker front-runs or races the transaction by calling `enforceJail()` for a single operator in the batch.
3. When `enforceJailMany()` executes, the loop reaches that operator and reverts with `AlreadyEnforced`.
4. The entire batch reverts, preventing jailing of all remaining operators.
5. Repeated attempts can be griefed in the same way, forcing jailing to be done serially and wasting gas.

Recommendation: Consider adding a pre-check in `enforceJailMany()` to skip operators that have already been enforced before calling `enforceJail()`.

NIL-7
Unbounded Active Operator Set Can Cause Gas/Memory DoS During Committee Selection

• Low ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `869ab86`.
The client provided the following explanation:

> Enforced a max active-operator cap to prevent unbounded active set growth during committee selection.

File(s) affected: `contracts/src/StakingOperators.sol`, `contracts/src/WeightedCommitteeSelector.sol`

Description: The `StakingOperators` contract maintains an `_activeOperators` array with no hard upper bound. If `ProtocolConfig` is unset or configured with a very low `minOperatorStake` (including zero, which still allows any operator with stake > 0), an attacker can create many Sybil operator addresses, stake the minimum amount, and register them as active.

In `WeightedCommitteeSelector` the `selectCommittee()` function calls `stakingOps.getActiveOperators()` and loads the entire active-operator array into memory before applying the `maxActiveOperators` cap. Although the cap limits the number of operators ultimately considered, it does not limit the cost of reading and iterating over the full array. This can cause committee selection to become increasingly expensive or revert due to gas or memory limits, resulting in a denial of service.

Recommendation: Consider enforcing a hard upper bound on the active-operator set or require a sufficiently high `minOperatorStake` at all times.

NIL-8
Slashing Callback Can Block Round Finalization via `OOG`/return-data Bomb

• Informational ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `f743293`.
The client provided the following explanation:

> Slashing callback is now performed with a gas cap and ignored return data to prevent OOG/return-data blocking of finalization.

File(s) affected: `contracts/src/HeartbeatManager.sol`

Description: A malicious or buggy slashing policy can cause the external call in `_notifySlashing()` to consume all gas or return oversized data, which is not catchable by `try/catch` and reverts the whole transaction, preventing round finalization and stalling progress

Recommendation: Use a low-level call with an explicit gas cap and ignore return data (e.g., `address(policy).call{gas: GAS_LIMIT}(...)`) or move slashing notification to a separate, non-blocking transaction so the slashing policy cannot revert finalization.

NIL-9
Staker Binding Cannot Be Cleared when Operator Has Zero Stake and No Tranches

• Informational ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `011901a`.
The client provided the following explanation:

> Clear operator-staker binding when stake is zero even if no unbonding tranches exist.

File(s) affected: `contracts/src/StakingOperators.sol`

Description: The `operatorStaker` mapping is cleared only when `StakingOperators.withdrawUnstaked()` empties all unbonding tranches and the operator's stake reaches zero. However, `StakingOperators.withdrawUnstaked()` reverts if no unbonding tranches exist. Therefore, an operator that has been fully slashed and has no remaining tranches cannot clear the staker binding, leaving a stale operator–staker state.

Recommendation: Consider adding an explicit code path to clear the staker binding when an operator's stake is zero, even if no unbonding tranches are present.

NIL-10
Unbonding Tranche Limit May Prevent Further Unstake Requests

• Informational ⓘ Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> Addressed in: `8755d63`.
> The client provided the following explanation:
>
> > Documented unbonding tranche limit and operational requirement to withdraw matured tranches.

File(s) affected: `contracts/src/StakingOperators.sol`

Description: The contract enforces a maximum of `MAX_TRANCHES_PER_OPERATOR = 32` active unbonding tranches per operator. Once this limit is reached, subsequent `StakingOperators.requestUnstake()` calls revert with `TooManyTranches()`. Because `unstakeDelay` can be as long as 365 days, operators who repeatedly request partial unstakes without withdrawing matured tranches may be unable to submit additional unstake requests for an extended period, even though they still have a bonded stake.

Recommendation: Clearly document the unbonding tranche limit and its operational implications, including the need to withdraw matured tranches to free capacity for future unstake requests.

NIL-11
`RewardPolicy` Is Incompatible with Deflationary Tokens

• Informational ⓘ Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> Addressed in: `ce89e5f`.
> The client provided the following explanation:
>
> > Documented requirement that reward token is a standard ERC20 without transfer fees or rebasing

File(s) affected: `contracts/src/RewardPolicy.sol`

Description: `RewardPolicy` assumes the reward token balance only changes due to explicit transfers and that `balanceOf(address(this))` accurately reflects deposited and owed funds. If a deflationary or fee-on-transfer token is used, transfers into or out of the contract may reduce the actual received amount, causing `accountedBalance` to diverge from the real balance. This can trigger accounting underflow, freeze the contract (`accountingFrozen`), or break reward distribution and withdrawals.

Recommendation: Document and enforce that `rewardToken` must be a standard ERC20 without transfer fees or supply-modifying mechanics.

NIL-12 Unsafe Casting Can Cause Integer Truncation

• Informational ⓘ Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `f743293`.
> The client provided the following explanation:
>
> > Added SafeCast usage for responseWindow and jailDuration snapshotting to prevent unsafe truncation.

File(s) affected: `contracts/src/HeartbeatManager.sol`

Description: There are multiple instances where an integer is casted unsafely. When the cast moves from a larger integer type to a smaller integer type, the integer can become truncated when it is greater than the maximum of the smaller integer type. Although in the described cases this is unlikely to happen as these are timestamps, they are nonetheless set by privileged roles and can thus cause unintended consequences.

This occurs in `HeartbeatManager._snapshotRoundConfig()` in:

1. `config.responseWindow()` (`uint256`) is casted to a `uint64`
2. `config.jailDuration()` (`uint256`) is casted to a `uint64`

Recommendation: Consider using OpenZeppelin's `SafeCast` library when casting data types.

Auditor Suggestions

S1 Remove Unchecked Arithmetic

Mitigated

Update

Marked as "Fixed" by the client.
Addressed in: `10a3ab1` .
The client provided the following explanation:

> Upgraded pragma and removed redundant unchecked loop increments.

Update

Marked as "Mitigated" since according to the Solidity release notes, the counter "may not be modified in the loop condition or the loop body". Therefore in order to get the benefit of automatic `unchecked ++i` , we recommend putting the incrementation at the end of the `for` clause instead of the loop body.

Description: The codebase contains instances of unchecked arithmetic, specifically for loop counter increments. Since Solidity version 0.8.22, the compiler automatically optimizes simple loop counter increments without requiring explicit `unchecked` blocks. Consider using a newer Solidity version; these manual optimizations are redundant and add unnecessary complexity to the code.

Recommendation: Use a newer Solidity version (0.8.22 or newer) and remove all instances of `unchecked { i++; }` or similar constructs in loops throughout the codebase.

S2 Code Duplication

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `414ede5` .
The client provided the following explanation:

> Deduplicated HeartbeatManager vote digest and finalize logic to reduce duplication.

File(s) affected: `contracts/src/HeartbeatManager.sol`

Description: Duplicate code can cause increased contract bytecode size, limiting the possibility of future feature additions. It also increases the gas cost of deployment. Instances where duplicate code is found:

- `HeartbeatManager`
  - `escalateOrExpire()`
    - At lines 386 and 390, the function `_finalizeRound()` is called with the same arguments at the start of either branch of a conditional. Hence both calls can be merged into one call before the beginning of the conditional.
  - `_verifyVoteSig()` / `voteDigest()`
    - At lines 295-6 and 305-6, the same logic is used in both cases.

Recommendation: Remove duplicate code.

S3 Code Improvements

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `11939ba` .

The client provided the following explanation:

> Applied code-quality improvements (events, constants, dead code removal, and stream-rate optimizations).

Description: The following improvements are proposed:

1. Fixed In `RewardPolicy`, the `onlyHeartbeatManager` modifier is invoked only once. Consider removing the modifier and inlining the logic into the calling function instead.
2. Fixed In `StakingOperators`, the `snapshot()` function updates `currentSnapshotId` but does not emit an event. Consider emitting an event to enable off-chain indexers to track the changes.
3. Fixed In `HeartbeatManager`, the `_computeCommitteeSize()` function uses the magic number `10_000`. Consider defining and using a constant variable instead of a literal value. If the same literal value is used multiple times, create a constant state variable and reference it throughout the contract.
4. Fixed In `HeartbeatManager`, the `distributeRewards()` function includes state variable changes but does not emit an event. Consider emitting an event to enable off-chain indexers to track the changes.
5. Fixed In `RewardPolicy.accrueWeights()`, when `distributed == 0`, a second full iteration over `recipients` is used to select the fallback recipient (highest weight, lowest address), even though this information can be tracked during the initial loop that validates ordering and computes `totalWeight`. Consider tracking the highest-weight recipient during the first iteration in `accrueWeights()` and reuse it in the fallback path to avoid redundant looping.
6. Fixed In `RewardPolicy._recomputeStreamRate()`, the `epochDuration == 0` branch is unreachable under normal operation because `epochDuration` is never allowed to be zero, resulting in dead code. Consider removing or explicitly document the unreachable `epochDuration == 0` branch in `_recomputeStreamRate()` to reduce dead code and improve maintainability.
7. Fixed In `HeartbeatManager._startRound()`, the second `snapshotId == 0` branch is unreachable because the `else` branch only executes when the outer condition `snapshotId == 0` is false. Consider removing the second unreachable `snapshotId == 0` to reduce dead code and improve maintainability.

Recommendation: Consider applying the code improvements as described.

S4 Missing Validation

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `0794ffb`.
> The client provided the following explanation:
>
> > Added protocol configuration validations, including contract checks, bounds checks, and duration limits.

File(s) affected: `contracts/src/WeightedCommitteeSelector.sol`, `contracts/src/StakingOperators.sol`, `contracts/src/ProtocolConfig.sol`, `contracts/src/HeartbeatManager.sol`

Description: The following validations are proposed:

1. Fixed In `WeightedCommitteeSelector`, when updating the `minCommitteeVP` state variable in the `setMinCommitteeVP()` function or the constructor, it is not validated for a zero amount.
2. Fixed In `StakingOperators`, when updating the `protocolConfig` state variable in the `setProtocolConfig()` function, it does not validate that the address is a contract and implements any of the functions in the `IProtocolConfig` interface.
3. Fixed In `HeartbeatManager`, the `setConfig()` function is missing validation that the `newConfig` address is a contract and implements at least one function defined in the `IProtocolConfig` interface.
4. Fixed In `ProtocolConfig`, there is missing validation that the addresses passed in `setModules()` are contract addresses.
5. Fixed In `ProtocolConfig`, there is missing validation that the `_committeeSizeGrowthBps` passed in `setParams()` is a valid basis points value (`<= 10,000`).
6. Fixed In `ProtocolConfig`, there is missing validation that `quorumBps_` is greater than zero.
7. Fixed In `ProtocolConfig`, there is missing validation that `verificationBps_` is greater than zero.
8. Fixed In `ProtocolConfig`, there is missing validation that `responseWindow` is greater than zero.
9. Fixed In `ProtocolConfig`, there is missing validation that `jailDuration` is greater than zero.

Recommendation: Consider applying the validations as described.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.

- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/NillionNetwork/niluv`

- `70c...536 ./contracts/src/EmissionsController.sol`
- `c88...945 ./contracts/src/HeartbeatManager.sol`
- `a07...778 ./contracts/src/Interfaces.sol`
- `e42...439 ./contracts/src/JailingPolicy.sol`
- `3a1...86a ./contracts/src/NoOpSlashingPolicy.sol`
- `411...7c1 ./contracts/src/ProtocolConfig.sol`
- `b92...e21 ./contracts/src/RewardPolicy.sol`
- `e85...8c3 ./contracts/src/StakingOperators.sol`
- `56a...bf4 ./contracts/src/WeightedCommitteeSelector.sol`

Test Suite Results

```
forge test
```

Tests appear to be comprehensive and are all passing.

```
Ran 4 tests for test/EmissionsController.t.sol:EmissionsControllerTest
[PASS] test_emissionForEpoch_bounds() (gas: 19216)
[PASS] test_globalCapExceeded_reverts() (gas: 1014532)
[PASS] test_mintAndBridge_enforcesEpochTiming() (gas: 177710)
[PASS] test_mintAndBridge_isPermissionless() (gas: 136020)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 262.21ms (19.59ms CPU time)

Ran 11 tests for test/StakingOperators.t.sol:StakingOperatorsTest
[PASS] test_fullUnstakeStillClearsBinding() (gas: 308188)
[PASS] test_jail_deactivates_and_requires_reactivate() (gas: 512201)
[PASS] test_registerOperator_revertsWhenActiveSetFull() (gas: 714345)
[PASS] test_register_requiresMinStake() (gas: 523615)
[PASS] test_requestUnstake_and_withdraw() (gas: 472316)
[PASS] test_slash_active_and_unbonding() (gas: 469101)
[PASS] test_snapshot_and_stakeAt_checkpoints() (gas: 458492)
[PASS] test_stakeTo_allowsTopUpAfterMinIncrease() (gas: 340501)
[PASS] test_stakeTo_bindsStaker() (gas: 348808)
[PASS] test_stakeTo_requiresMinStakeOnInitialBind() (gas: 104399)
[PASS] test_stakeTo_revertsForUnauthorizedStakerWhenApproved() (gas: 341559)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 262.50ms (14.39ms CPU time)

Ran 6 tests for test/RewardPolicy.t.sol:RewardPolicyTest
```

```
[PASS] test_accrueWeights_revertsOnLengthMismatch() (gas: 210785)
[PASS] test_accrueWeights_revertsOnUnsortedRecipients() (gas: 212012)
[PASS] test_claim_transfersAndClears() (gas: 304943)
[PASS] test_commitmentMismatch_afterInitialRevert() (gas: 333942)
[PASS] test_dustDistribution_allocatesOneWei() (gas: 291831)
[PASS] test_maxPayoutCap() (gas: 359106)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 262.76ms (21.38ms CPU time)

Ran 4 tests for test/ProtocolConfig.t.sol:ProtocolConfigTest
[PASS] test_constructor_revertsOnInvalidBps() (gas: 63167)
[PASS] test_constructor_revertsOnZeroModules() (gas: 61877)
[PASS] test_setModules_onlyOwner() (gas: 1129640)
[PASS] test_setParams_validatesCommitteeCaps() (gas: 772666)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 263.25ms (58.98ms CPU time)

Ran 1 test for test/HeartbeatFlow.t.sol:HeartbeatFlowTest
[PASS] testFullHeartbeatFlow() (gas: 972173)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 307.68ms (8.81ms CPU time)

Ran 1 test for test/HeartbeatFlow.t.sol:HeartbeatInvalidRewardFlowTest
[PASS] testInvalidThresholdRewardsInvalidVoters() (gas: 2066232)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 309.12ms (8.56ms CPU time)

Ran 1 test for test/HeartbeatManagerCommitteeGrowth.t.sol:HeartbeatManagerCommitteeGrowthTest
[PASS] test_committeeSize_grows_on_escalation() (gas: 2250664)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 12.86ms (5.13ms CPU time)

Ran 4 tests for test/JailingPolicy.t.sol:JailingPolicyTest
[PASS] test_enforceJailFromMembers_jailsNonvoters_and_incorrectVoters() (gas: 1722610)
[PASS] test_enforceJailFromMembers_revertsOnRootMismatchOrUnsorted() (gas: 1265860)
[PASS] test_enforceJail_individualWithProof() (gas: 1478811)
[PASS] test_roundIsRecordedOnFinalize() (gas: 1237580)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 316.24ms (15.60ms CPU time)

Ran 5 tests for test/WeightedCommitteeSelector.t.sol:WeightedCommitteeSelectorTest
[PASS] test_largeCommitteeSelection() (gas: 73419614)
[PASS] test_selectCommittee_capsToMaxActiveOperators_whenPoolIsHuge() (gas: 565909648)
[PASS] test_selectCommittee_fallsBackToPrevrandao() (gas: 1650319)
[PASS] test_selectCommittee_respects_maxActiveOperators_pool() (gas: 6289102)
[PASS] test_selectCommittee_unique_and_bounded() (gas: 6229845)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.17s (939.26ms CPU time)

Ran 28 tests for test/HeartbeatManager.t.sol:HeartbeatManagerTest
[PASS] test_abandonRewardDistribution_blocksPayout_and_is_ownerOnly() (gas: 1272731)
[PASS] test_committeeRoot_matchesOffchainMerkleComputation() (gas: 845982)
[PASS] test_distributeRewards_revertsOnCountMismatch() (gas: 1446954)
[PASS] test_distributeRewards_revertsOnInconclusiveOutcome() (gas: 1426420)
[PASS] test_distributeRewards_revertsOnInvalidVoterInList() (gas: 1455322)
[PASS] test_distributeRewards_revertsOnUnsortedVoters() (gas: 1447989)
[PASS] test_distributeRewards_revertsOnWeightMismatch_invalidOutcome() (gas: 1269469)
[PASS] test_doubleVote_reverts() (gas: 941861)
[PASS] test_escalateOrExpire_beforeDeadline_reverts() (gas: 825904)
[PASS] test_escalateOrExpire_inconclusive_startsNewRound_and_then_expires() (gas: 1564901)
[PASS] test_escalateOrExpire_revertsOnRawHashMismatch() (gas: 838703)
[PASS] test_finalize_invalidThreshold_updatesStatus() (gas: 1259603)
[PASS] test_finalize_validThreshold_updatesStatus() (gas: 1240173)
[PASS] test_moduleUpgrade_doesNotAffectExistingRoundRewardAddress() (gas: 3071124)
[PASS] test_pause_blocks_submitHeartbeat_and_votes() (gas: 849125)
[PASS] test_retrySlashing_retriesAfterFailure() (gas: 1346786)
[PASS] test_retrySlashing_revertsWhenNotFinalized() (gas: 823805)
[PASS] test_setSlashingGasLimit_ownerOnly_and_nonZero() (gas: 27024)
[PASS] test_submitHeartbeat_burnsBondOnInvalidOutcome() (gas: 1264089)
[PASS] test_submitHeartbeat_idempotent_noSecondRoundStarted() (gas: 821694)
[PASS] test_submitHeartbeat_refundsBondOnValidOutcome() (gas: 1245890)
[PASS] test_submitHeartbeat_revertsForInvalidSnapshotId() (gas: 328544)
[PASS] test_submitHeartbeat_startsRoundAndSetsPending() (gas: 846167)
[PASS] test_submitVerdict_revertsAfterDeadline() (gas: 857558)
[PASS] test_submitVerdict_revertsForNonMember() (gas: 826004)
[PASS] test_submitVerdictsBatched_hardLimit500() (gas: 580631)
[PASS] test_submitVerdictsBatched_respects_configMaxBatch() (gas: 955994)
[PASS] test_submitVerdictsBatched_signature() (gas: 979091)
Suite result: ok. 28 passed; 0 failed; 0 skipped; finished in 40.80s (89.10ms CPU time)
```

```
Ran 3 tests for test/invariants/HeartbeatManagerInvariants.t.sol:HeartbeatManagerInvariants
[PASS] invariant_committeeTotalStakeMatchesSumStakeAtSnapshot() (runs: 256, calls: 128000, reverts: 0)

╭-----------------+------------------+-------+---------+----------╮
| Contract        | Selector         | Calls | Reverts | Discards |
+=================================================================+
| HeartbeatHandler | escalateIfNeeded | 42647 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | vote             | 42578 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | warp             | 42775 | 0       | 0        |
╰-----------------+------------------+-------+---------+----------╯

[PASS] invariant_roundAccountingSums() (runs: 256, calls: 128000, reverts: 0)

╭-----------------+------------------+-------+---------+----------╮
| Contract        | Selector         | Calls | Reverts | Discards |
+=================================================================+
| HeartbeatHandler | escalateIfNeeded | 42519 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | vote             | 42773 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | warp             | 42708 | 0       | 0        |
╰-----------------+------------------+-------+---------+----------╯

[PASS] invariant_voteWeightsMatchSnapshotStake() (runs: 256, calls: 128000, reverts: 0)

╭-----------------+------------------+-------+---------+----------╮
| Contract        | Selector         | Calls | Reverts | Discards |
+=================================================================+
| HeartbeatHandler | escalateIfNeeded | 42639 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | vote             | 42546 | 0       | 0        |
|-----------------+------------------+-------+---------+----------|
| HeartbeatHandler | warp             | 42815 | 0       | 0        |
╰-----------------+------------------+-------+---------+----------╯

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 47.76s (126.13s CPU time)

Ran 5 tests for test/BlacklightSystem.t.sol:BlacklightSystemTest
[PASS] test_endToEnd_valid_workflow_rewards_and_claim() (gas: 18650892)
[PASS] test_jailing_removesFromNextCommitteeSelection() (gas: 21068724)
[PASS] test_largeCommittee_200_members_finalizes_and_jailing_enforcement() (gas: 171400772)
[PASS] test_largeCommittee_weighted_rewards_track_stake_across_rounds() (gas: 74234198)
[PASS] test_veryLargeOperatorPool_reward_distribution_and_jailing() (gas: 579746065)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 60.57s (794.45ms CPU time)

Ran 4 tests for test/invariants/StakingInvariants.t.sol:StakingInvariants
[PASS] invariant_activeSetHasNoDuplicatesAndIsConsistent() (runs: 256, calls: 128000, reverts: 0)

╭---------------+----------------+-------+---------+----------╮
| Contract      | Selector       | Calls | Reverts | Discards |
+================================================================+
| StakingHandler | deactivate     | 12923 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | jail           | 12836 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | poke           | 12652 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | reactivate     | 12690 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | register       | 12756 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | requestUnstake | 12799 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | slash          | 12822 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | stake          | 12916 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
| StakingHandler | warp           | 12814 | 0       | 0        |
|---------------+----------------+-------+---------+----------|
```

```
| StakingHandler | withdraw       | 12792 | 0       | 0        |
└----------------+----------------+-------+---------+----------┘

[PASS] invariant_operatorStakerZeroImpliesNoStakeAndNoUnbonding() (runs: 256, calls: 128000, reverts: 0)

┌----------------+----------------+-------+---------+----------┐
| Contract       | Selector       | Calls | Reverts | Discards |
+================================================================+
| StakingHandler | deactivate     | 12844 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | jail           | 12652 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | poke           | 12796 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | reactivate     | 12781 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | register       | 12667 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | requestUnstake | 12895 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | slash          | 12995 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | stake          | 12660 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | warp           | 12781 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | withdraw       | 12929 | 0       | 0        |
└----------------+----------------+-------+---------+----------┘

[PASS] invariant_totalStakedEqualsSumOfActiveAndUnbondingAcrossTrackedOperators() (runs: 256, calls:
128000, reverts: 0)

┌----------------+----------------+-------+---------+----------┐
| Contract       | Selector       | Calls | Reverts | Discards |
+================================================================+
| StakingHandler | deactivate     | 12927 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | jail           | 12783 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | poke           | 12819 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | reactivate     | 12714 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | register       | 12806 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | requestUnstake | 12824 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | slash          | 13015 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | stake          | 12687 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | warp           | 12562 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | withdraw       | 12863 | 0       | 0        |
└----------------+----------------+-------+---------+----------┘

[PASS] invariant_totalStakedMatchesTokenBalance() (runs: 256, calls: 128000, reverts: 0)

┌----------------+----------------+-------+---------+----------┐
| Contract       | Selector       | Calls | Reverts | Discards |
+================================================================+
| StakingHandler | deactivate     | 12925 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | jail           | 12831 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | poke           | 12735 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | reactivate     | 12763 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | register       | 12728 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | requestUnstake | 12777 | 0       | 0        |
```

```
|----------------+----------------+-------+---------+----------|
| StakingHandler | slash          | 12831 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | stake          | 12763 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | warp           | 12837 | 0       | 0        |
|----------------+----------------+-------+---------+----------|
| StakingHandler | withdraw       | 12810 | 0       | 0        |
└----------------+----------------+-------+---------+----------┘

Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 60.57s (133.09s CPU time)

Ran 13 test suites in 60.64s (212.87s CPU time): 77 tests passed, 0 failed, 0 skipped (77 total tests)
appuser@dd54e6803efe:~/workspace/projects/AT-3205-Nillion/niluv-fixes/contracts$
```

Code Coverage

```
forge coverage --no-match-coverage "mock|libraries"
```

Test coverage not available due to stack too deep compiler errors.

```
Error: Compiler run failed:
Error: Compiler error (/solidity/libsolidity/codegen/LValue.cpp:51):Stack too deep. Try compiling with `--via-ir` (cli) or the equivalent `viaIR: true` (standard JSON) while enabling the optimizer. Otherwise, try removing local variables.
  --> script/DeployEmissionsController.s.sol:53:13
```

Changelog

- 2026-01-12 - Initial report
- 2026-01-16 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1100 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republicc, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

Quantstamp

© 2026 – Quantstamp, Inc.